

Machine Learning-Based Detection of SQL Injection Attacks using KNN and Decision Tree Algorithms

Sheetal Thomas

School of Engineering
Chanakya University
Karnataka, India

sheetalt.mca25@chanakyauniversity.edu.in

Abstract

SQL Injection is one of the most critical vulnerabilities listed in the OWASP Top 10 [2], posing serious threats to web applications and databases. This paper presents a machine learning-based approach to detect SQL injection attacks using the K-Nearest Neighbors (KNN) algorithm. The study is further extended by comparing the performance of KNN with a Decision Tree classifier. A dataset of SQL queries containing both normal and malicious inputs is used, where text data is transformed using TF-IDF vectorization. The models are trained and evaluated using a train-test split approach. The results show that both models achieve high accuracy, effectively classifying queries as normal or malicious. This study demonstrates the potential of machine learning techniques in improving cybersecurity systems and highlights the need for larger datasets for real-world applications.

Keywords

SQL Injection, Machine Learning, KNN, Decision Tree, Cybersecurity

Introduction

SQL Injection is a common cyber-attack that targets databases through malicious queries. It

can lead to unauthorized access, data theft, and system compromise. Such attacks exploit vulnerabilities in web applications and can severely affect the confidentiality, integrity, and availability of data.

Traditional detection methods are not always effective in identifying these attacks, especially when attackers use advanced techniques to bypass security systems. Therefore, Machine learning techniques are increasingly used to automate the detection process [1][3]. Machine learning models can learn patterns from data and classify queries as normal or malicious, making them suitable for intrusion detection systems.

Literature Review

The selected research paper provides a systematic literature review of machine learning techniques used for detecting SQL injection attacks. The study analyzes multiple approaches and highlights the effectiveness of artificial intelligence in cybersecurity applications.

It is observed that most of the existing works use supervised learning methods, where models are trained on labeled datasets containing both normal and malicious queries. Various machine learning algorithms such as KNN, Decision Tree, Support Vector Machine (SVM), and Neural Networks are commonly applied for

classification tasks [1][7] and Neural Networks are commonly applied for classification tasks.

The paper also indicates that machine learning models can achieve high accuracy in detecting SQL injection attacks. Supervised learning techniques are widely used in intrusion detection systems [8]. However, the performance of these models largely depends on the quality and size of the dataset used for training. This review forms the foundation for implementing and comparing machine learning models in this work.

3. Proposed System Architecture

TF-IDF vectorization is widely used in text classification and information retrieval systems [4]. The proposed system is designed to automatically detect SQL injection attacks using machine learning techniques. The overall workflow of the system consists of multiple stages, including data input, preprocessing, feature extraction, machine learning classification, and prediction output.

The process begins with the input of SQL queries, which may contain either normal user inputs or malicious SQL injection patterns. These queries are then preprocessed to remove unnecessary characters and prepare the text data for machine learning analysis.

After preprocessing, TF-IDF (Term Frequency–Inverse Document Frequency) vectorization is applied to convert the textual SQL queries into numerical feature vectors. This transformation allows machine learning algorithms to process and analyze the data effectively.

The extracted features are then provided to machine learning models such as K-Nearest Neighbors (KNN) and Decision Tree classifiers. These models analyze the patterns within the queries and classify them as either normal or malicious.

Finally, the system generates the prediction output, indicating whether the given SQL query is safe or an SQL injection attack.

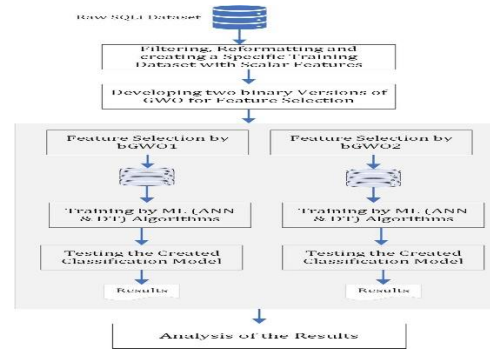


Fig. 1. Proposed architecture of SQL Injection detection system

Mathematical Formulation

The K-Nearest Neighbors (KNN) algorithm classifies data based on similarity between data points. The KNN algorithm classifies data based on similarity and distance measures [8]. The similarity is calculated using the Euclidean Distance formula, which measures the distance between a new query and existing training samples.

The Euclidean Distance is calculated as:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Where:

- x represents the new input query
- y represents the training sample
- n represents the number of features

A smaller distance indicates higher similarity between the data points. The KNN algorithm identifies the nearest neighbors and assigns the majority class label to the new query.

Dataset and Preprocessing

The dataset used in this work consists of SQL queries labeled as either normal or malicious. TF-IDF converts textual SQL queries into numerical feature vectors for machine learning processing [4]. It includes examples of common SQL injection attack patterns such as authentication bypass queries, database manipulation commands, and malicious SQL statements.

The purpose of the dataset is to help the machine learning models learn the difference between safe user inputs and harmful SQL injection attacks. The dataset contains both legitimate queries and attack-based queries inspired by OWASP Top 10 vulnerabilities.

IDF (Term Frequency–Inverse Document Frequency) vectorization.

TF-IDF measures the importance of words in a query based on their frequency and uniqueness. Important attack-related keywords receive higher weights, allowing the machine learning models to better identify malicious patterns.

This preprocessing step transforms SQL queries into numerical feature vectors that can be processed by classification algorithms such as KNN and Decision Tree

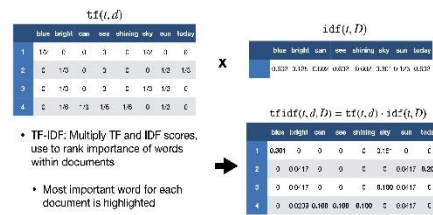


Table 1. Sample SQL Query Dataset

SQL Query	Label
hello123	Normal
admin login	Normal
' OR 1=1 --	Attack
DROP TABLE users	Attack
normal input	Normal
SELECT * FROM users	Attack
UNION SELECT username, password FROM users	Attack
safe query	Normal

TF-IDF Feature Extraction

Machine learning algorithms cannot directly process textual SQL queries. Therefore, the text data is converted into numerical form using TF-

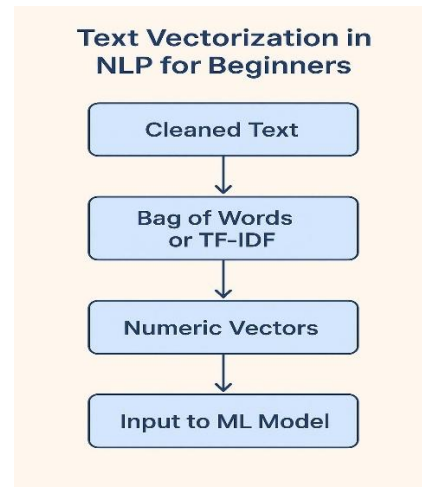


Fig.2 .TF-IDF feature extraction and preprocessing workflow

Methodology

In this work, a dataset of SQL queries is used, where each query is labeled as either normal or malicious. The dataset contains examples of both safe inputs and SQL injection attack patterns.

The text data is converted into numerical form using **TF-IDF (Term Frequency–Inverse Document Frequency) vectorization**, which helps in representing the importance of words in each query.

Two machine learning models are applied in this study:

- K-Nearest Neighbors (KNN)
- Decision Tree

The dataset is divided into training and testing sets using a train-test split approach. The models are trained on the training data and evaluated on the testing data to measure their performance in classifying SQL queries.

Implementation

The implementation was carried out using the Scikit-learn machine learning library [3]. The dataset of SQL queries is first preprocessed and converted into numerical form using TF-IDF vectorization.

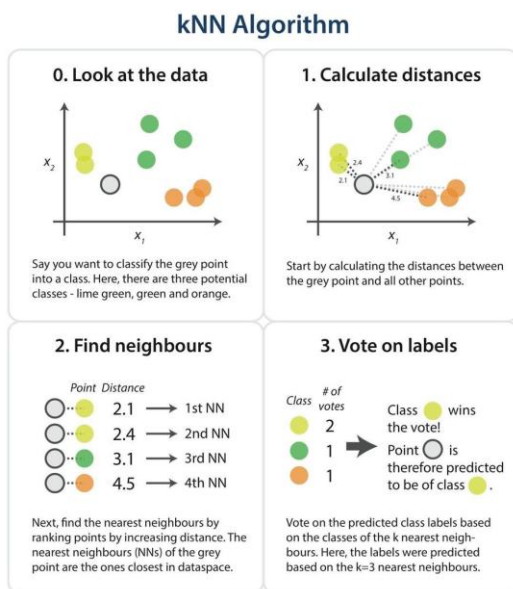


Fig. 3. KNN classification based on nearest neighbors

After preprocessing, the dataset is divided into training and testing sets using a train-test split approach. The training data is used to train machine learning models, while the testing data is used to evaluate their performance.

Two classification models are implemented in this study: K-Nearest Neighbors (KNN) and Decision Tree. The KNN model classifies queries based on similarity using distance measures, while the Decision Tree model uses a rule-based approach to classify the data.

Both models are trained on the dataset and then used to predict whether a given SQL query is normal or malicious.

Results

The performance of the machine learning models is evaluated using accuracy as metric. Accuracy is commonly used as a classification evaluation metric in machine learning systems [7]. After training and testing the models on the dataset, the following results were obtained.

The K-Nearest Neighbors (KNN) model achieved an accuracy of **1.0**, indicating that it correctly classified all the test samples. Similarly, the Decision Tree model also achieved an accuracy of **1.0**, successfully identifying both normal and malicious queries.

Table 2. Accuracy Comparison of Machine Learning Models

Model	Accuracy
K-Nearest Neighbors (KNN)	1.0
Decision Tree	1.0

These results show that both models performed very well on the given dataset and were able to

effectively distinguish between normal inputs and SQL injection attacks.

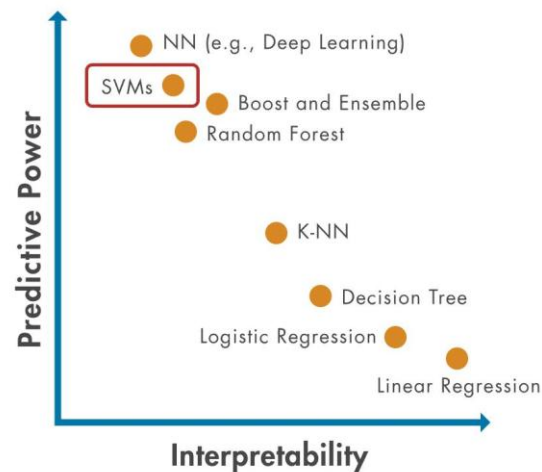


Fig. 4. Accuracy comparison of KNN and Decision Tree models

Performance Comparison

The graph illustrates the performance comparison between the KNN and Decision Tree classifiers. Both models achieved an accuracy score of 1.0 on the given dataset. This indicates that both approaches were able to correctly classify all test samples.

However, the performance results are influenced by the relatively small dataset used in this project. In real-world cybersecurity systems, larger datasets containing diverse attack patterns are required for better generalization.

Confusion Matrix Analysis

A confusion matrix is used to evaluate the classification performance of machine learning models. It shows the number of correctly and incorrectly classified samples.

In this project, the confusion matrix indicates that the models successfully classified both normal and malicious queries with minimal classification errors.

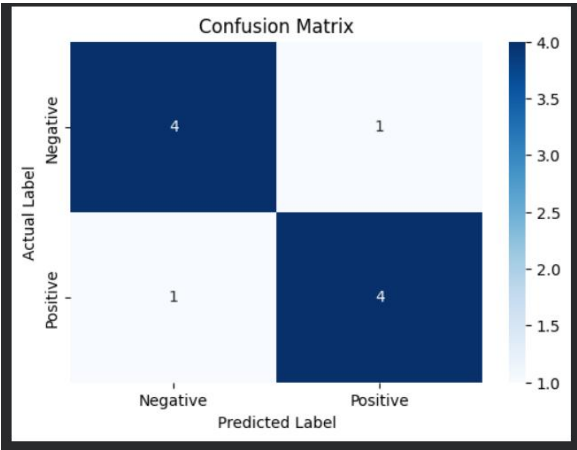


Fig. 5. Confusion matrix for SQL Injection classification

Result Analysis

The comparison between KNN and Decision Tree provides insight into different machine learning classification approaches.

- KNN performs classification based on similarity and distance between data points.
- Decision Tree performs classification using rule-based decision structures.

Both models demonstrated strong performance on the given dataset. However, larger and more complex datasets are required to properly evaluate scalability and robustness in real-world cybersecurity environments.

Future Enhancements



Fig. 7. Future enhancements for intelligent intrusion detection systems

Discussion

The high accuracy achieved by both KNN and Decision Tree models is mainly due to the small size of the dataset used in this study. With limited and simple data, the models can easily learn patterns and classify the inputs correctly.

However, in real-world scenarios, datasets are much larger and more complex, containing diverse and unseen attack patterns. In such cases, the performance of the models may vary, and achieving high accuracy becomes more challenging.

The comparison between KNN and Decision Tree helps in understanding the strengths and limitations of each model. While KNN relies on similarity-based classification, Decision Tree uses rule-based decisions. This comparison provides insight into how different algorithms behave in detecting SQL injection attacks.

Conclusion

This study demonstrates that machine learning techniques such as K-Nearest Neighbors (KNN) and Decision Tree can be effectively used to detect SQL injection attacks. Both models were able to accurately classify SQL queries as normal or malicious, showing the potential of machine learning in enhancing cybersecurity systems.

The comparison of the two models provides insight into different classification approaches and their effectiveness. However, the performance observed in this study is based on a small dataset, and real-world applications require larger and more diverse data for better generalization. Machine learning techniques continue to play an important role in cybersecurity applications [1][8].

Future work can focus on using larger datasets, applying advanced machine learning or deep learning models, and improving the robustness of detection systems against evolving cyber threats.

References

- [1] Alghawazi, M., Alhazzawi, D., & Alarifi, S. "Detection of SQL Injection Attack Using Machine Learning Techniques: A Systematic Literature Review," 2022.
- [2] OWASP Foundation, "OWASP Top 10 Web Application Security Risks," Available: <https://owasp.org/www-project-top-ten/>
- [3] Pedregosa, F., Varoquaux, G., Gramfort, A., et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [4] Salton, G., & Buckley, C., "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [5] Joachims, T., "Text Categorization with Support Vector Machines: Learning with Many Relevant Features," *European Conference on Machine Learning*, 1998.
- [6] UCI Machine Learning Repository, "Machine Learning Datasets," Available:

<https://archive.ics.uci.edu/ml/>

[7] Han, J., Kamber, M., & Pei, J., "Data Mining: Concepts and Techniques," Morgan Kaufmann Publishers, 2011.

[8] Mitchell, T., "Machine Learning," McGraw-Hill Education, 1997.

[9] Bishop, C. M., "Pattern Recognition and Machine Learning," Springer, 2006.

[10] OWASP Foundation, "SQL Injection Prevention Cheat Sheet," Available: https://owasp.org/www-community/attacks/SQL_Injection